Write-up: Model Documentation

1. Who are you?

● Name (first and last name or first name and last initial): TANG YONGWEI(唐永威)
● Hometown: ZHEJIANG,CHINA
● Social handle or URL (optional):https://xhslink.com/m/A9w2jw56Xg0

My profession is algorithm engineer, and I have won multiple championships and runner-up prizes in various competitions such as machine learning, deep learning, and large models.

2. Motivation

The generous prize money and the matching ASR problem.

3. High-Level Approach

I found that the leaderboard score and local validation score differed significantly, so my main approach was experience-driven. I chose Qwen3-ASR-1.7B as the base model and implemented inference acceleration.

4. Visualizations

No, mainly based on loss and validation set scores.

5. Three Most Impactful Code Segments

Part 1: valid_df=train_df[train_df['split']!=fold].reset_index(drop=True)

train_df=train_df[train_df['split']==fold].reset_index(drop=True)

I originally split the training set, but later I used the entire training set and that brought me directly to first place in the early stage. Part 2: from IPython import get_ipython
get_ipython().system(f'''torchrun --nproc_per_node=4 qwen3_asr_sft.py
--model_path data/models/X4090D-v013/checkpoint-85950/
--train_file data/feature/train_{version}.jsonl
--eval_file data/feature/valid_{version}.jsonl
--output_dir data/models/{version}
--batch_size 3
--grad_acc 1

--lr 1e-6
--epochs 5
--log_steps 10
--save_strategy steps
--save_steps 28650
--save_total_limit 5
--num_workers 8
--pin_memory 1
--persistent_workers 1
--prefetch_factor 2''') data/models/X4090D-v013/checkpoint-85950/ is the checkpoint from my 13th experiment. Here I fine-tuned it further with a learning rate of 1e-6. This operation improved the public leaderboard by only 0.002, but improved the private leaderboard significantly. Part 3: model = Qwen3ASRModel.LLM("/code_execution/src/checkpoint-143250", gpu_memory_utilization=0.9, max_inference_batch_size=32, # Batch size limit for inference. -1 means unlimited. Smaller values can help avoid OOM. max_new_tokens=4096, # Maximum number of tokens to generate. Set a larger value for long audio input.) Here Qwen3ASR is launched with vLLM. If vLLM is not used for inference, even a 0.6B model would time out, let alone 1.7B.

6. Machine Specs & Time

- CPU (model): Intel(R) Xeon(R) Gold 6148 CPU @ 2.40GHz
- GPU (model or N/A): RTX 4090D
- Memory (GB): 300
- OS: Ubuntu
- Train duration: 60 hour
- Inference duration: 1 hour

7. Caveats & Known Issues

During inference I load all audio files into memory in parallel, so you need to ensure sufficient memory.

8. Tools for Data Preparation / EDA

No.

9. Performance Evaluation Beyond Competition Metric

I mainly split the validation set by child ID. Compared to random splitting, this can show, to some extent, the model's generalisation ability across different children.

10. Things Tried That Didn't Make Final

Because the maximum inference time limit was 2 hours, I trained Qwen2.5-omni-7B and Kimi-Audio-7B separately to infer on 5% of the audio, leaving the remaining 95% to Qwen3-ASR-1.7B. I hoped to improve accuracy by using larger models, but in the end both models performed worse on the test set, even though they performed well on the validation set.
I also collected more child data to train Qwen3-ASR-1.7B, adding 80k audio samples, but performance on the test set was still poor.
I added random noise to the audio to improve robustness, but this operation not only hurt WER, but also hurt noise WER.

11. Future Improvements

Because the maximum inference time limit was 2 hours, I trained Qwen2.5-omni-7B and Kimi-Audio-7B separately to infer on 5% of the audio, leaving the remaining 95% to Qwen3-ASR-1.7B. I hoped to improve accuracy by using larger models, but in the end both models performed worse on the test set, even though they performed well on the validation set.
I also collected more child data to train Qwen3-ASR-1.7B, adding 80k audio samples, but performance on the test set was still poor.
I added random noise to the audio to improve robustness, but this operation not only hurt WER, but also hurt noise WER.

12. Simplifications for Faster Inference

When optimising the inference speed of Kimi-Audio-7B, I found that making the model support batch inference is actually faster than vLLM.